Analog and Digital Communications

2nd Year Electronic Laboratory

Imperial College London

Version 2.00

Prepared by:

S. Somuyiwa – D. Kurka – T. Tze-Yang – S. Sreekumar - M. Mohammadi Amiri – J. Puyol-Roig

C. Leung – D. Gunduz

Lab 1: Introduction to LabView

The purpose of this lab is to introduce you to the LabView Communications System Design Suite's environment. In this first lab, you will learn how to implement mathematical functions and create basic communications modules.

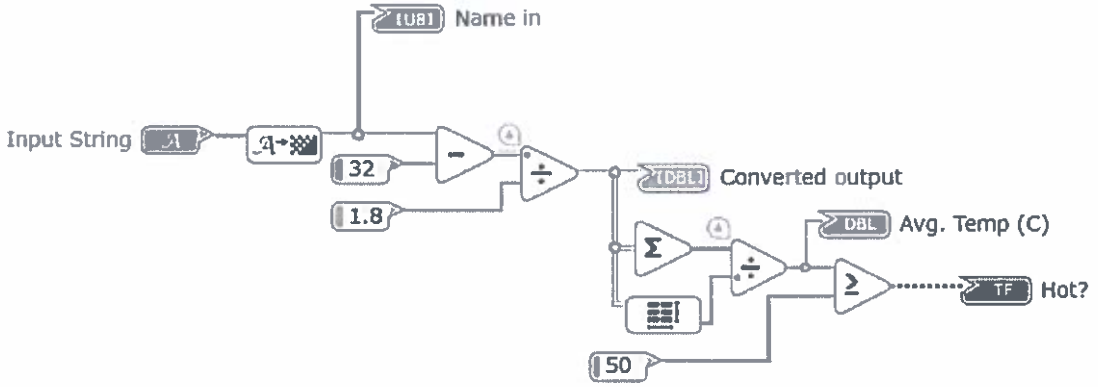
Exercise 1: Data Types in LabView

In this exercise you will examine the data types in LabView, and do some basic conversions between them. The purpose of the code will be to convert each letter of a string into its ASCII binary representation, which will be shown in their equivalent unsigned integer form. Then by treating each integer as a Fahrenheit temperature, you will convert it into Celsius. Finally, an average of the Celsius values will be taken and compared to a threshold value to indicate if the string you inputted is considered "Hot".

Instructions:

1. From the "**File**" menu, create a new "**Blank Project**", and name it **Lab1**.
2. On the "**Files**" pane, add a new VI, and name it **DataTypes.gvi**.
3. Now you will add the module through which you can input strings. This can be done from the "**function palette**" under the "**Diagram tab**". Using the search box, search for the module **String to byte array** and create a control from the input by right clicking on the terminal "string" and selecting "create control". This will allow you to input a string from the Panel tab. You can also learn more about the modules by clicking on the module, and on the right panel click on "**Context Help**".
4. Create an indicator at the output of the **String to byte array** module by right clicking on the "unsigned byte array" terminal and choosing "Create indicator". This will allow you to view the byte array on the Panel tab in the form of unsigned integers representing the ASCII values of each character of the string you input.
5. Assuming that each ASCII value represents a temperature in Fahrenheit, convert the array of integers into Celsius values. Arithmetic operations can be applied to whole arrays by connecting the array and a scalar constant to an operator node. **Hint:** *You can right click on the terminals of a module to create constants, and if it gives you an array constant you can switch to a scalar constant by right clicking on the constant itself.*
6. Create an indicator for the converted array.
7. Calculate the average of the converted values using the modules **Add Array Elements** and **Array Size**. Create an indicator showing the average value.
8. Apply a threshold by using the module **Greater or Equal**, and compare the average value to a constant. Set the constant to 50 and create an indicator called "**Hot?**" at the output of the **Greater or Equal** module.
9. Test the code by going to the Panel tab and placing all the controls and indicators. Input your name into the string control and view the result by pressing the play button.

***Hint**: You can change the name of any block by double-clicking on the name.*



Task:

☞ Input your surname on "input string" field and retrieve the integer representation of its first three characters within a single execution of the program. Copy (take a screenshot) the block diagram and the front panel of your code, and include them in your logbook.

Exercise 2: Implementation of the Central Limit Theorem

In this exercise you will visualize the central limit theorem by using loop structures, graphical blocks and arrays. Independent uniformly distributed random numbers will be generated, and the convergence of the sample mean value to a normal distribution will be observed as the number of trials increases. You should create and save a new VI called "**CLT.gvi**" under your project Lab1.

In a nutshell, we will generate a vector of independent and identically distributed random numbers (using a for loop), evaluate their sample mean, and iterate this within a while loop. After each iteration, we will update the histogram of the mean values to observe the convergence of the distribution.

Instructions:

1. In order to observe the random experiment continuously, place a **While Loop** (in Program Flow) to the block diagram. The [i] symbol on the lower left corner of the loop contains the value of the current iteration of the loop. You will also see a conditional terminal (the red button) created on the lower right corner of the while loop, which allows you to terminate the loop if a certain condition is met. To enter the desired stopping condition, right-click on the input terminal of the conditional terminal, and choose "Create control". A stop button will also be available on the front panel.
2. Inside the loop, place a timing module called **Wait**, which will control the breaks between each execution of the while loop. To determine the duration of breaks, create a control for this node. Remember to place the corresponding control in the front panel.
3. Place a **For Loop** inside the while loop. As an equivalent of the nested loops in textual programming languages, place a second **For Loop** within the last one which will allow you to create a 2D array.
4. Adjust the number of iterations of both **For Loops** by placing constants connected to the left-hand side loop count terminal ([N] symbol). This will allow you to define the dimensions of the 2D array you will create.
5. Search for the **Add Array Elements** module, and place the function outside the nested **for** loops. This will allow you to sum the elements of the generated array, which is necessary for calculating the sample mean.
6. Inside the **For Loops**, place a **Random Number Generator,** and connect its output to the **Add Array Elements** function. Notice the boxes that are created on the frames of **For Loops**. These boxes are called **Tunnels,** and they specify how the data will be transferred through the frame. By default, these tunnels are indexing tunnels (right click on them and ensure auto-indexing is selected). An auto-indexing output tunnel appends the single value obtained from a single loop iteration to an accumulating array of data.
7. Now, in order to find the mean value, we need to divide the sum of the generated random values to the size of the random array. Find the **Divide** function in the block diagram and connect the output of **Add Array Elements** to the "X" terminal of it. Then right click on the "Y" terminal and create a constant. The constant should be equal to the total number of elements of the array (product of the number of rows and the number of the columns), so that the output of the division operation is the sample mean of your random array.
8. You need to create an array of these sample means in order to create a histogram. Start by placing **Insert into Array function** on the block diagram. This function inserts new elements into a specific position of an input array, and also has the ability to increase the size of the array. Connect the sample mean you calculated to the "New Element/Subarray" terminal.
9. Connect the [i] symbol of the while loop (loop iteration) to the "Index" terminal of **Insert into**

Array function. This will assign the new mean value generated to its corresponding position within an array storing all results.

10. Now you need a base array to add all these new elements to, and you need it to be available at each iteration. Here a new concept called "**Shift Registers**" will be introduced. Shift register is a type of tunnel that carries information from an iteration to the next one. First, connect the output array of **Insert into Array function** to the right frame of the while loop. Then right click on the tunnel and select "Replace with Shift Register". This will create another tunnel on the left edge of the loop. Connect this new tunnel to the "array" terminal of the **Insert into Array function**. The tunnel on the left edge carries the data from the previous iteration, whereas the one on the right edge sends the data to the next iteration. Finally, right click on the "input" terminal of the left register, and create a **constant array** so it can initialise the program with a 0 array.



11. Your loop is complete, and all you need is to create and observe the histogram. Search for the **Histogram function** and connect the output of the **Insert into Array function** to the "signal" terminal of the **Histogram Function**. Then create a control for "number of bins" terminal that will allow you to control the number of bins in the histogram.
12. Finally, to observe the changes in the histogram, create an indicator to the histogram terminal. You should create the indicator within the while loop. Add the corresponding indicator graph in the front panel
13. Switch to front panel, set "milliseconds to wait" to 10, "number of bins" to 100, and run the code.

Tasks:

☞ Include the block diagram and the front panel in your logbook.

☞ Create a control for stopping the execution of the program after 1000 iterations. Set the value of "intervals" to 10. Include the corresponding histogram in your logbook.

☞ Revise your code such that the final distribution you observe in the histogram is a standard Normal distribution (i.e., it has zero mean and unit variance). Include the revised block diagram in your logbook, and explain each of the changes you have made.

Exercise 3: Waveforms

In this exercise, you will learn how to implement signal generation, filters and waveforms in LabView. Please start with creating a new VI for this experiment.

Instructions:

1. First of all, you need to generate a signal from the **Wave Generator** module, which can be found by searching in the function palette. It generates a sine waveform by default.
2. For the adjustments of the generated signal, you should create the controls for "frequency", "amplitude", "sample rate" and "samples" terminals. Import two more "Sine Waveform" modules to generate three signals, and add these three signals together.
3. To observe the power spectrum of the combined waveform, search for the **FFT Power Spectrum and PSD** module. Insert it to the diagram and change its configuration to "Power Spectral Density".
4. Connect the combined waveform to the "signal" input of the module. To observe the output of the FFT function, a graph from the panel is needed. Create the graph first on the panel,

and then connect the created graph node in the block diagram to the PSD output terminal of the **FFT Power Spectrum and PSD** module.

Hint: *To zoom-in on the graph, click on the graph on the panel page, and on the configuration pane (right) select "Graph Tools" under the Parts section. Now you should see a toolbar underneath your graph, with which you can zoom in/out.*

5. To eliminate the higher frequency components of your signal, a low pass filter should be applied. Use the **Filter** module and select "Lowpass" and "Butterworth" in the panel on the right to configure it as a lowpass Butterworth filter. Connect the combined waveform to the signal input and place controls on the "order" and "cutoff frequency" terminals. Next, add a graph to the output of the filter to observe the filtered signal.
6. Assign the values in the table below to corresponding controls, and run the code.

Low cutoff frequency	10
Sampling Rate	1k
Number of Samples	1k
Order	5
Frequency_1	10
Frequency_2	30
Frequency_3	120
Amplitude	1

Tasks:

☞ Include the block diagram and the front panel in your logbook. The three output graphs, i.e., generated waveform, waveform PSD and filtered waveform should also be included.

☞ Modify the filter parameters such that you have only the second sine wave (one with frequency 30) at the output. Include these parameters in your logbook, and explain the reason why it works.

Analog and Digital Communications

2nd Year Electronic Laboratory

Imperial College London

Version 2.00

Prepared by:

S. Somuyiwa – D. Kurka – T. Tze-Yang – S. Sreekumar - M. Mohammadi Amiri – J. Pujol-Roig

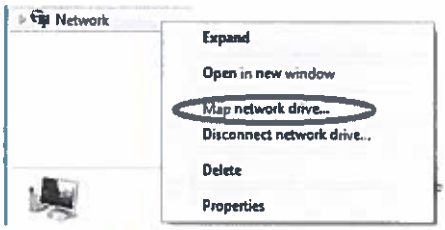
C. Leung – D. Gunduz

Lab 2: AM Simulation and USRP

In this lab you will build AM and FM communication system simulators. In order to achieve this, you will learn how to use sub-VIs in the LabView environment.

Open **LabVIEW Communications Design Suite** and create **a new project** called "**Lab2**". All the files created for solving the following exercises should be included in the Lab2 project.

Some files/VIs that will be used in this and subsequent labs have been provided to you. To access these files, right click on the "Network" icon under "Computer", and then click on "Map network drives" (as shown in the figure on the right). In the window that opens, type the following address in the "Folder" field: **\\ic.ac.uk\group\ee_labs\commslabs**. Then click Finish.



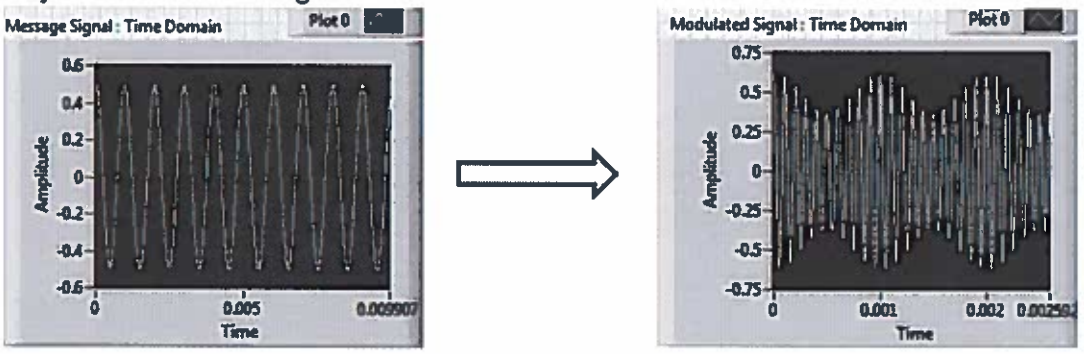
Note: To use any of these files/VIs in any project, download the specific files/VIs from the server and save them in the same folder as the project. If you refresh the Files pane in Labview, with the project open, you will see these files/VIs appear under the project.

Exercise 1: AM Modulator

In the first part of this lab you are going to build an AM modulator. Recalling the theory of AM, the standard equation for modulating a single tone is given by:

$$s(t) = [A_c + A_m \cos(2\pi f_m t)]\cos(2\pi f_c t)\,,$$

where A_m and A_c are the amplitudes, and f_m and f_c are the frequencies of the message signal and the carrier waveform, respectively. An example of the application of this modulation is shown in the following figure, with the message signal given on the left and the modulated signal on the right. The envelope of the modulated signal is generated by the (shifted) message signal that has a lower frequency than the carrier signal.



Instructions:

1. Create a new VI called "**AM.gvi**".
2. Create two sine waveforms by using the "**Sine Waveform**" module in the Diagram window. These functions will be used to generate the message and carrier waveforms. Create and populate the values of the necessary terminals of the waveform generators according to the following table.

	Message Signal	**Carrier Signal**
Frequency	1kHz (Control)	10kHz (Control)
Phase	90° (Constant)	90° (Constant)
Amplitude	Message Amplitude (Control)	Carrier Amplitude (Control)
Sample rate	200k (Control)	200k (Control)
Samples	1000 (Control)	1000 (Control)

3. Obtain the power spectrum of the message signal (remember the waveform exercise in Lab 1). Create graphs in the panel to show the PSD of the message signal, and the waveform plots (in time domain) of both the message and the carrier signals.

4. Perform the necessary operations in order to obtain the AM waveform.
5. As you did in step 3, create an indicator to show the AM waveform and the AM power spectrum.
6. You have now obtained the AM modulated signal. The **modulation index** is defined as $\mu = \frac{A_m}{A_c}$. You can observe the effect of the modulation index on the modulated waveform by changing the values of A_m and A_c via the corresponding controllers in the panel window.
7. The LabView Communications System Design Suite allows users to pack their VI's into **sub-VIs,** which make it easier to include them in other VIs. This also makes your block diagram look neater, and creates a modular code that can be modified easily. In the following steps we will explore this functionality.
    - First, click on the **Edit Icon** button located on the right hand side of the top banner. Now you can click on the various terminals to configure them either as an *input* (left) or an *output* (right). In this example, the **inputs** are: message signal frequency, message signal amplitude, carrier signal frequency, carrier signal amplitude, sampling info; while the **outputs** are: message signal, carrier signal and AM waveform time domain representations, and message signal and AM waveform PSDs.
    - Once saved your VI is ready for future use as a sub-VI.

*Hint: Explore the **Cleanup Diagram** tool (✦), on the top banner of the diagram tab, to make your diagram look neater.*

> ***Hint:** If necessary, you can adjust the number of terminals on each side of the icon by dragging and expanding the icon itself. Once you connect the necessary inputs and outputs you can use the palette on the left hand side of the screen to design a suitable image for the icon of your GVI.*

Tasks:

☞ Adjust all the plots you retrieve so that they are clear and easy to understand.

☞ Include the block diagram in your logbook.

☞ Set $A_c = 2$ and change A_m in the panel window to obtain the modulation index values $\mu = \{0.5, 1, 1.5\}$. For each value of μ plot the time domain and PSD representations of both the message and modulated signals.

☞ What is the impact of the modulation index on the modulated signal? Explain your answer.

☞ Set $A_c = 1$ and $A_m = 1$. Change the frequency of the message signal to $f_m = \{1k, 2k, 5k\}$. What is the impact of the varying frequency value on the modulated signal? Explain your answer.

Exercise 2: AM Demodulators

Create a demodulation module to recover the original message signal. There are two possible methods:

1. **Coherent Demodulation**: Multiply the AM signal with the carrier signal followed by a low pass filter.
2. **Envelope Detection**: Rectify the AM signal, low pass filter it and remove the DC component to obtain the envelope.

Create two VI modules **'AMCoherent.gvi'** and **'AMEnvelopeDetection.gvi'** for the two different types of demodulation techniques.

Exercise 2a: Coherent Detection

Instructions:

1. Open the "AMCoherent.gvi" file.
2. Perform AM signal and carrier signal multiplication:
    a. Add the **Multiply** function to the diagram.

b. Create two controls by searching for "Waveform Array Terminal" and right click on the function and select "Change to Scalar". Name them as "AM signal" and "carrier", respectively. Their corresponding graphs will appear on the panel window.

3. Connect the output of the multiplication to a **Butterworth Filter** (recall from Lab 1). Create controls for the **order** and **Low Cut-off Freq** of the filter.
4. Subtract the DC component from the main signal. The DC component can be retrieved by using the **Amplitude Measurements** module, and in the right panel configuring the module to perform "AC DC and RMS" and "Single Shot".
5. Add waveform graph indicators to view the final results in both time and frequency domains. (When you test this module later with an input signal, you will be able to observe the message signal in both time and frequency domains after demodulation).

Hint: *Pay attention to appropriately scaling the amplitude in order to get the desired message signal.*

Handling Errors in Labview

LabView allows controlling the errors that may occur for different reasons during the execution of a code. Just as data values flow through a VI, so does the error information, and this is the reason why most functions include error input and output terminals. If LabVIEW detects an error, the node passes the error to the next node. The next node does the same thing, and so on. At the end of the execution flow, LabVIEW reports the error. Thus, by wiring the error information from the beginning of the GVI to the end, and by setting appropriate indicators, it is possible to obtain details on the errors in the Panel tab.

In particular, the following functions can be useful in the following exercises:

a. When handling multiple errors, the **Merge Errors** function (available from the function palette) allows merging multiple error clusters into a single one, which can thus be propagated till the end of the function itself.
b. If you need to convert an error code to an error cluster, you will need the **Build Error Cluster** module in the function palette.

It is a good practice to properly handle errors when writing a GVI. Even if it is not a necessary condition in order to obtain the right result.

6. Transform your VI into a sub-VI as in Exercise 1 (Inputs: AM signal, carrier signal, low cut-off frequency, order, and error in. Outputs: demodulated signal in time, demodulated signal PSD, error out).

Tasks:

☞Add the block diagram to your logbook.

☞Explain briefly the mathematical theory behind this demodulation technique. Moreover, why are we using a low pass filter and why do we have to get rid of the DC component? Why do you need to scale the message amplitude?

Adjusting Plots with Labview

When a new plot is added to LabView, a default graph is created, adjusted automatically to the input data. However, in order to visualise and analyse data, further steps may be needed.

1. **Zooming and panning** – after selecting the graph in the panel, tick the **Graph Tools** option on the right hand panel under "Parts". A menu with zooming and panning options should appear under the graph. 
2. **Renaming axes** – Every graph shows "time" and "amplitude" as default axis labels (independently of the input data). To change the names to something meaningful, just double click on the axis name.
3. **Exporting and visualizing data** – Right click a plot and select **"Capture Data"**. This will export the current data to the "Data" menu (left of the screen) where you can visualize the data in a bigger screen and save it. It is also possible to drag captured data to a graph in the panel, to visualize it.
4. Many other settings can all be found under the right panel.

Exercise 2b: Envelope Detection

This method consists of a **half wave rectifier** and an **RC filter.**

Instructions:

1. Open the 'AMEnvelopeDetection.gvi' file.
2. Since the multiplication with the impulse response of the rectifier introduces a loss of π in the signal, we need to amplify it before rectification by the same amount. Thus, multiply the signal with π.
3. To implement the rectifier, convert the waveform to an array using the **Waveform Properties** function, then use a **for** loop that runs through all the elements of the AM signal, and selects only the positive values (set the negative values to 0). Thus, inside the **for** loop, you will need to put a condition that checks if an element is positive (have a look at the **Case Structure** or the **Select** function under "Program Flow" in the functions palette).
4. Convert the array (of the rectified AM signal) back to a waveform using the **Build Waveform** function.
5. Build a low pass filter using the **Butterworth Filter** with the same values used in Exercise 2a. Also remove the DC component.
6. Transform your VI into a sub-VI as in Exercise 1 (Inputs: AM signal, order, low cut-off frequency, filter type, error in. Outputs: demodulated signal in time, demodulated signal PSD, error out).

Tasks:

☞ Explain briefly the mathematical theory behind this demodulation technique and all the steps implemented.

☞ Add the block diagram to your logbook.

Exercise 3: AM Simulation

In this exercise, all the sub-VIs you have prepared so far will be used, and you will be able to observe the entire process of amplitude modulation and demodulation.

Instructions:

1. First, create a new file and name it "**AMTopLevel.gvi**".
2. Drag **AM**, **AMCoherent** and **AMEnvelopeDetection** from the file tab on the left side of the screen, and place them on the block diagram.
3. Create necessary control and indicator terminals for each node, and make the necessary connections.

4. Create a while loop with a control that allows you to stop the loop via a stop button. Run the code and observe the process.

Tasks:

☞ Add the block diagram to your logbook.

☞ Set the parameters as in the following table and run the code.

Carrier amplitude	2
Message signal amplitude	See question 2
Sample frequency	200k
Samples	1k
Butterworth low-pass filters	Order: 5 Cut-off frequency: 3 kHz
Carrier frequency	10 kHz
Signal message frequency	1 kHz

☞ Observe the time domain and PSD representations of the coherent and envelope demodulated signals as the message signal amplitude changes from 1 to 4 with steps of 1. As you change the message signal amplitude do you observe any changes between the two modulation techniques? If yes, explain the reason.

Exercise 4: AM Communications via USRP

In this exercise, all the required modules have already been provided to you. Your task is to run the modules using the settings provided and explain how the modules work from the code.

Some background information on the modules concerning the USRP:



niUSRP Open Tx Session initiates the USRP device, generates a session handle and an error cluster that are propagated through all four VIs. When you use this VI, you must add a control called "device name" that you will use to inform LabView about the USRP Device ID (either a name or an IP address).



niUSRP Configure Signal is used to set the parameter values. Attach four controls and three indicators to this VI as shown in Figure 1. The controls set the **IQ rate** (in Sample/s), the **carrier frequency**, the **gain**, and the **active antenna**. When the VI runs, the radio returns the actual values of these parameters (called "coerced parameters"). The actual parameters may or may not match the values put as controls. If an input value exceeds the capability of the radio, it will choose the nearest acceptable parameter value, as the coerced value, rather than returning an error.



niUSRP Write Tx Data sends the baseband signal to the USRP for transmission. Placing this VI in a while loop allows a block of baseband signal samples to be sent over and over until the "stop" button is pressed. Note that the while loop is programmed to terminate if an error is detected. Baseband signal samples can be provided to the **Write Tx Data** either as an array of complex numbers, or as a complex waveform data type. Right click on the function and select **Replace**. This will allow you to choose the data type of the function. The input data should be complex in order to introduce I and Q data to the USRP (which corresponds to the real and imaginary parts, respectively). If the baseband discrete time signal is expressed as

$$\tilde{g}[nT_x] = g_I[nT_x] + jg_Q[nT_x]$$

then the continuous time transmitted signal from the USRP is

$$g(t) = Ag_I(t)\cos(2\pi f_c t) - Ag_Q(t)\sin(2\pi f_c t)$$

In this expression, the constant A is set by the "gain" parameter (in dB) and f_c is the carrier frequency. The sampling interval T_X is the reciprocal of the "IQ rate". Be aware that the USRP internal circuit implements some functions for transmitting and receiving the signal (Appendix A contains details of the USRP's internal operations). The signal g(t) produced by the USRP is a continuous time signal, that is, digital to analog conversion (DAC) is done inside the USRP itself.



niUSRP Close Session terminates transmitting operation once the while loop ends. Note that the VI should be terminated using the STOP button rather than with "Abort Execution" on the toolbar, so that the Close Session VI will correctly close the data structures that the VI uses



niUSRP Open Rx Session works in the same way as **niUSRP Open Tx Session.**



niUSRP Configure Signal works in the same way as in the Tx.



niUSRP Initiate sends the parameter values you selected to the receiver, and starts it running.



niUSRP Fetch Rx Data retrieves the message samples received by the USRP. Placing this VI in a while loop allows message samples to be retrieved one block at a time until the "stop" button is pressed. **Number of samples** control allows you to set the number of samples retrieved at each step of the while loop. **Fetch Rx Data** can provide message samples to the user as either an array of complex numbers, or as a complex waveform data type. Right clicking and then selecting **Replace** allows you to choose the data type for the message samples. This function does not output the received signal directly. Instead, *USRP itself multiplies the received signal with the carrier, based on the configurations you set in the panel control, and applies a low pass filter to it* (see Appendix A).



niUSRP Abort stops the acquisition of data once the while loop ends.



niUSRP Close Session works in the same way as in the Tx.

Instructions:

1. Open the files named "**USRP_AM_Rx.gvi**" and "**USRP_AM_Tx.gvi**".
2. In the front panel of each module, set the values of the controls as in the following:

Control	Tx	Rx
Device names	NI2900	NI2900
IQ rate	500k	500k
Carrier frequency	400M	400M
Gain	0	0
Active antenna	TX1	RX2
Sample rate	500k	N/A
Samples	200k	200k

Low cutoff frequency	N/A	1400
Modulation index	1	N/A
Message frequency	1000	N/A

Tasks:

☞ Run the code and explain in your logbook how the transmitter and receiver work. **Hint:** the USRP does all the modulation and demodulation on its own so the modules only needs to provide the complex data for the USRP to modulate. At the receiver the USRP will return the demodulated data.

☞ Transmit a single-tone message signal with frequency 5kHz and modulation index 1 (you may have to adjust the low cutoff frequency of the filter). Run the receiver, and plot the demodulated signal in time, and its PSD. Adjust the time and PSD plot to the relevant region so that the plot is not just a blur, and rename the axes appropriately. Add the plots to your logbook.

☞ To observe the effect of noise in the demodulated signal, increase the receiver's gain to 20 dB (your receiver will start to detect other weaker signals in addition to the transmitted signal), and adjust the X axis of the demodulated message (in time domain) to show values between 0 second (s) and 0.004 s. Then, change the modulation index (μ) value and observe the effects on the plots of the demodulated signal in both time and frequency. From what value of μ can noise be clearly noticed in the plots? Copy the image of the noisy demodulation into your logbook.

Exercise 5: Listening to AM Music (Bonus)

Inside the lab, there is a USRP broadcasting an AM signal. The signal is a piece of music that you can listen to.

Instructions:

1. Set the USRP_AM_Rx_Music.gvi panel setting to the configuration below, and try to find the message signal.

***Hint**: LabView might have problems detecting the right soundcard settings automatically to play your audio. **Before running the VI**, plug-in the earphones, and try to play any audio file first (e.g. a Youtube video), preferably with LabView closed. This will ensure that the right soundcard setting is selected.*

Control	Rx
Device names	NI2900
IQ rate	100k
Carrier frequency	400M
Gain	Adjust as necessary
Active antenna	RX1
Samples	5M

Note: *LabView requires all the samples to be acquired before playing the signal, hence it may take some time before you hear the first note. Once the output signal graph starts changing, it indicates the signal is being played, and you should hear the music.*

Task:

☞ Add the graph of the demodulated signal both in time and frequency domain to your logbook and identify the frequency of the message signal and the song playing.

Analog and Digital Communications

2nd Year Electronic Laboratory

Imperial College London

Version 2.00

Prepared by:

S. Somuyiwa – D. Kurka – T. Tze-Yang – S. Sreekumar - M. Mohammadi Amiri – J. Puyol-Roig

C. Leung - D. Gunduz

Lab 3: FM Simulation and USRP

Exercise 1: FM Modulator

In FM modulation, information about the message signal is contained in the frequency of the modulated carrier waveform. The generalized function for FM signal is:

$$g(t) = A_c \cos\left(2\pi f_c t + \theta_m(t)\right),$$

where

$$\theta_m(t) = 2\pi k_f \int_0^t m(\tau)d\tau,$$

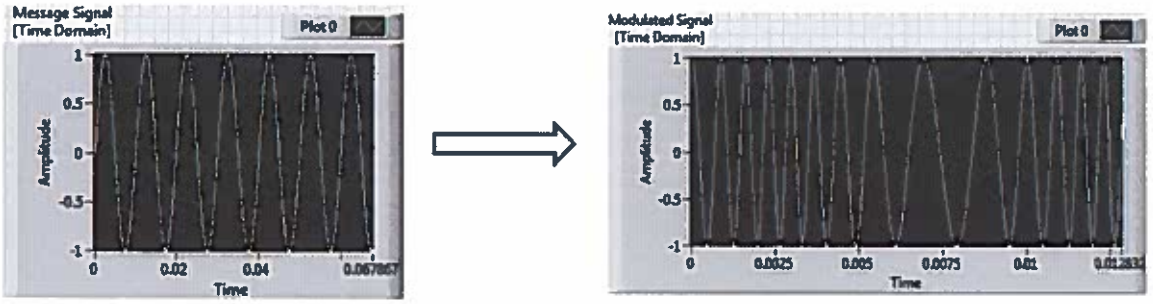
$m(t)$ is the message signal, and k_f represents the *frequency sensitivity*. The instantaneous frequency is:

$$f_i = f_c + k_f m(t)$$

In particular, we will use the following equivalent form:

$$s(t) = A_c \cos\left(2\pi f_c t\right)\cos\left(\theta_m(t)\right) - A_c \sin\left(2\pi f_c t\right)\sin\left(\theta_m(t)\right)$$

In the following figure, you can see an example of a message signal on the left and an FM modulated waveform on the right. Note the change in the frequency of the modulated signal in time.



The frequency deviation is defined as $\Delta_f = k_f m_f$, where $m_f = max|m(t)|$, and the modulation index (i.e., deviation ratio) is defined as

$$\mu = \frac{\Delta_f}{W},$$

where W is the message bandwidth.

Instructions:

1. Create a new project called "Lab 3" and a new module called "**FM_Modulator.gvi**".
2. In "**FM_Modulator.gvi**", generate a single tone message signal, and then using the "**Waveform Properties**", "**Integral x(t)**", "**Sine and Cosine**", and "**Build waveform**" modules, create the FM signal in its equivalent form as shown above.
3. Create indicators for the message signal and the output FM signal in time and frequency domains.

Tasks:

☞ Set the parameters as in the following table. Change Δ_f and observe the changes in the modulated signal. Add the graphs of the FM signal and its PSD to your logbook for Δ_f=500.

Message signal frequency	1 kHz
Carrier signal frequency	10 kHz
Sample rate	200k
Samples	1k
k_f	[500, 2000,5000]
Message signal amplitude	1
Carrier signal amplitude	1

☞ Explain the resulting changes in the FM signal and its PSD as you change Δ_f.

☞ Include the block diagram of your FM_Modulator.gvi in your logbook.

Exercise 2: FM Demodulator

One way of demodulating the FM signal is by means of the so-called *frequency discrimination*, which is obtained by:

1. Transferring the changes in instantaneous frequency into changes in the amplitude of the output signal, by differentiating the FM signal. The output of the differentiator is an AM+FM modulated signal.
2. Demodulate the AM+FM signal by using an envelope detector (as in Exercise 2b).

Instructions:

1. Create a new VI and name it "**FMDemodulator.gvi**".
2. Apply the **Derivative x(t)** found in the function palette on the signal.
3. After the differentiation, the remaining process is very similar to *envelope detection* in AM demodulation. Use exactly the same modules, and you will obtain the message signal. Remember to scale the amplitude appropriately.
4. Create a sub-VI and produce 3 outputs: demodulated signal in time, demodulated signal PSD, and error out.

Tasks:

☞ Explain briefly the mathematical theory behind this demodulation technique.

☞ Add the block diagram to your logbook.

Exercise 3: FM Simulation

In this part all the sub-VI's that you have prepared will be used in a single VI, and you will be able to observe the entire process of FM modulation and demodulation.

Instructions:

1. Create a new file and name it "**FMTopLevel.gvi**".
2. Add all the necessary VIs, and make the necessary connections.
3. As in Exercise 3, include all of the structure in a while loop, and add a stop button.
4. Use the following values:

Message signal frequency	1kHz
Carrier signal frequency	10kHz
Sample frequency	200kHz
Samples	1k
Butterworth low-pass filters	Order: 5 Cut-off frequency: 1400Hz
Message signal amplitude	2
Carrier signal amplitude	2
k_f	1000

Tasks:

☞ Add the block diagram to your logbook.

☞ Set the parameters as in the above table. Observe the demodulated signal and attach the plots of the FM signal and the demodulated signal with their corresponding PSDs to your logbook (for the demodulated signal, please provide also a zoomed version of it, without the initial transient, so that it is easy to observe the amplitude of the message).

Exercise 4: FM Communications via USRP

In this exercise, FM communication via USRP will be implemented. Instead of building the structure from scratch you are now given a complete file "**FM-TxRx.gvi**". The structure should look familiar, but with only a slight difference: you only have one VI this time. This is no big deal since you could have also combined the AM communication VIs into a single one. LabView has the ability to execute two (or more) while loops simultaneously; and thus there is no difference between the two approaches.

Instructions:

1. Have a detailed look at the code and compare it with the FM receiver/transmitter developed in the previous exercises. Run the VI, adjusting the parameters to reasonable values, and make sure you are able to transmit and receive successfully..

Exercise 4a: Carson's rule

The bandwidth of an FM signal is difficult to calculate analytically. In the 1920s J.R. Carson provided a rule of thumb for approximating the bandwidth:

$$B_{FM} \cong 2(\Delta f + B),$$

where B_{FM} is the bandwidth of the FM signal, Δf is the frequency deviation of the signal, and B is the message bandwidth.

Instructions:

1. The spectrum shown in the "**FM spectrum**" graph is identical to the power spectrum of the actual transmitted FM signal, except that the carrier will appear at 0 Hz, with the lower sideband on the negative-frequency side, and the upper sideband on the positive-frequency side.
2. To obtain the classic textbook FM spectrum, set the message as a single tone at 1 kHz. Run the transmitter and obtain power spectra of the transmitted signal for frequency deviations of 1 kHz, 5 kHz, and 30 kHz.

Task:

☞ Add the power spectrum graph for one of these cases into your logbook. Be sure to scale the horizontal axis so that each spectrum is visible clearly. Annotate your spectra to show the FM signal bandwidth (Carson's rule).

Exercise 5: Listening to FM Radio Using USRP (Bonus)

In this step you are expected to detect and listen to local radio stations through your USRP.

Detecting stations:

1. Open **Find Radio Station.gvi** file provided. This basic code reveals the power densities of frequencies around the carrier frequency that is entered.
2. Set the carrier frequency to 90 M and the IQ rate to 20M. Examine the Frequency plot obtained. The center of the graph (0) represents the chosen carrier frequency (90M) and the bandwidth is proportional to the chosen IQ Rate (+/- 10M in this case).

Task:

☞ Capture the data of the plot to explore it. Zoom in and try to find peaks on the graph. Each peak represents an FM signal detected by the USRP antenna. Change the parameters in Step 2 if necessary, and copy the obtained graph together with the chosen values to your logbook. According to your graph which FM station has the strongest signal?

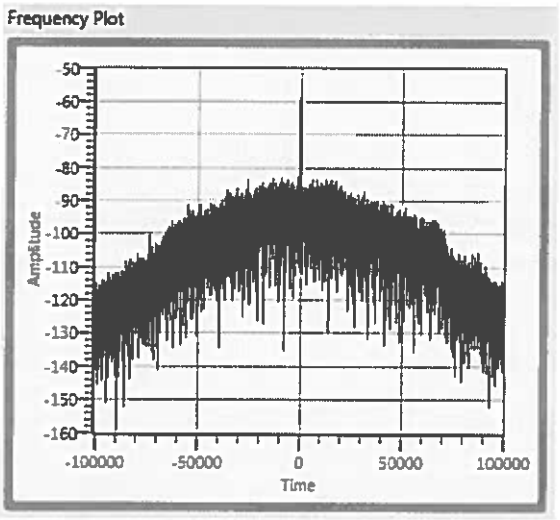


Figure 3: Example of a peak detected from spectrum analysis.

***Hint**: If you are unable to see peaks in the plotted data, try to increase the gain to 10, 25 and 30. The signal for FM radio stations can be weak inside the laboratory.*

***Hint**: If you get the 'overflow errors' message when running the VI, try to decrease the number of samples.*

Instructions:

1. From the analysis of the spectrum, choose an FM frequency that seems to have a strong signal. If the previous step failed, try 98.8 MHz – BBC 1.
2. Open **FM Music.gvi**. Look at the code, and observe that a module is added to play the demodulated waveform.
3. Input the chosen carrier frequency and a coherent bandwidth (IQ Rate) to tune into an FM radio station.

Task:

☞ Plug-in an earphone into the computer, turn the volume up and (hopefully!) enjoy the music!

***Hint**: When an earphone is plugged into the computer, LabView might have problems detecting the right soundcard settings automatically in order to play your audio. **Before running the VI**, plug-in the earphones, and try to play any audio file first (e.g. a Youtube video), preferably with LabView closed. This will ensure the right soundcard setting is selected.*

Analog and Digital Communications

2nd Year Electronic Laboratory

Imperial College London

Version 2.00

Prepared by:

S. Somuyiwa – D. Kurka – T. Tze-Yang – S. Sreekumar - M. Mohammadi Amiri – J. Puyol-Roig

C. Leung - D. Gunduz

Lab 4: Binary Phase Shift Keying (BPSK) Via USRP

In phase-shift keying (PSK) modulation, information is encoded on the phase of the transmitted carrier, rather than on its amplitude (ASK), or frequency (FSK). In binary phase-shift keying (BPSK) there are two phase values, 0^0 or 180^0, which means that an unmodified carrier is transmitted to represent one binary data value, while an inverted carrier is transmitted to represent the other. BPSK is optimum among binary modulation schemes in achieving the lowest average power for a given target bit error rate (BER)[1].

Exercise 1: BPSK Transmitter

A BPSK signal is a train of pulses, each of the form

$$Ag_{TX}(t)\cos(2\pi f_c t + \theta).$$

In this equation, A is a constant corresponding to the transmitted power level, $g_{TX}(t)$ is a fixed pulse shape, f_c is the carrier frequency, and θ takes the value of either 0^0 or 180^0 to carry the desired information. Note that, we can also write the equation as:

$$\pm Ag_{TX}(t)\cos(2\pi f_c t),$$

where the plus sign corresponds to $\theta = 0^0$, and the minus sign to $\theta = 180^0$. We will assume that a new pulse is transmitted every T seconds, so that the symbol rate (symbols per second) is $1/T$. For a binary scheme such as BPSK, the bit rate is the same as the symbol rate. Since the pulse $g_{TX}[n]$ does not carry information, its shape can be chosen to satisfy other criteria. We desire a pulse shape that provides a rapid spectral roll-off and minimizes inter-symbol interference, e.g. *'root-raised-cosine'* filter.

The steps needed to form a BPSK signal are:

1. Symbol Mapping: The input data arrives as a stream of bits. The **MT Generate Bits** VI produces an array of bits. For BPSK we will represent the bits in the following way:

Bit Value	Phase (θ)	Symbol
0	0	-1
1	180	+1

Remark: The USRP requires a complex valued input. Make sure you convert the symbols to complex numbers $(\pm 1 + j0)$ *before sending them as input to the USRP.*

2. Upsampling: As a first step towards replacing symbols with pulses, we will place $L-1$ zeros after each symbol. This produces a sample interval of

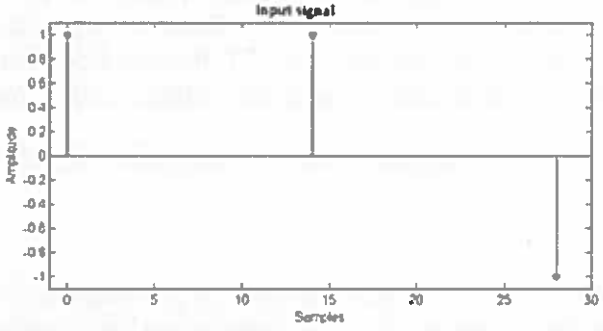
$$T_x = \frac{T}{L},$$

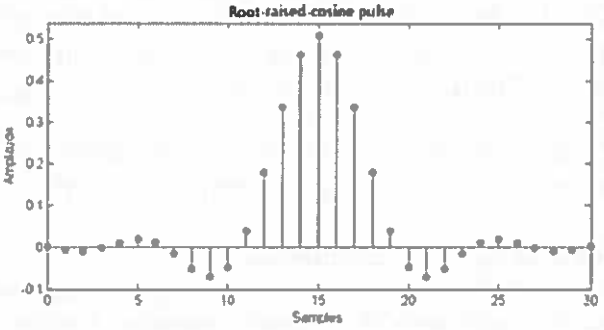
or, a sample rate of:

[1] BER represents the ratio of transmitted bits that have been incorrectly decoded at the receiver due to noise or interference. The goal is to achieve a BER as low as possible.

$$\frac{1}{T_x} = L\frac{1}{T}$$

3. Pulse Shaping: If the upsampled signal is applied to a filter whose impulse response $g_{TX}[n]$ is a root-raised-cosine pulse, then each symbol at the filter output will be represented by a root-raised-cosine pulse. This pulse shape has a very rapid spectral roll-off, so that the transmitted signal will not cause interference to signals at nearby carrier frequencies.





4. Modulation: The signal, consisting of a train of pulses of the form $(\pm 1 + j0)\, g_{TX}[n]$, can be sent directly to the USRP. The USRP transmitter will perform the DAC and the multiplication by the carrier $\cos(2\pi f_c t)$.

Following the steps below, construct a BPSK transmitter:
Note: *Templates have been provided in the file **BPSK_Tx_Rx_Template.gvi**. Note that the file contains both transmitter (upper) template and receiver (lower) template.*

1. Add an ***MT Generate* Bits (Fibonacci, PN Order)** from the function palette to your template. This will create a pseudorandom sequence of bits for your BPSK system. You can create a control to specify the total number of bits. Note that by default, **MT Generate Bits** will produce the same sequence of bits every time you run the code. This is useful for debugging, but if you would like to generate a different sequence of bits every time, connect a **random number** to the '*seed in*' input terminal.

Aligning the Received Bits
*In this experiment, we will compare the received bits with the transmitted bits and see how many bits have been correctly received. For this comparison, the bit sequence at the output of the receiver should align with the transmitted bit sequence. Therefore, the receiver must recognize the beginning of the transmitted sequence. The AddFrameHeader sub-VI inserts a specific 26-bit sequence at the start of the transmission. At the receiver, the **FrameSync** sub-VI looks for this specific sequence and cuts off all the bits received before this frame header, as well as the frame header itself.*

2. Do the symbol mapping, which will convert the integers 0 and 1 from **MT Generate Bits** to doubles of -1 and +1, respectively. Then add the **AddFrameHeader** function provided.
3. Upsample the array of symbols using the **Upsample** function from the palette. Set the symbol rate ($1/T$) to 10,000 symbols/s and the IQ rate ($1/T_x$) to 200,000 Samples/s. Use these two inputs to calculate the upsampling factor L.
4. Use **MT Generate Filter Coefficients** function from the palette to generate the pulse-shaping filter. Set the input terminals as follows:
    a. "Modulation Type" to PSK,
    b. "Pulse Shaping Samples per Symbol" to your calculated value of L,
    c. "Pulse Shaping Filter" to "Root Raised Cos"

5. Connect the "pulse shaping filter coefficients" output terminal to the "Y" input terminal of a **Convolution** function. Connect the output from your **Upsample** function to the "X" input of the **Convolution** function.
6. Normalize the amplitude of your filtered message signal to a maximum absolute value of 1. The provided **Quick Scale 1D** function finds the maximum of the absolute value.
7. To observe the baseband waveform, connect the output of the **Quick Scale 1D** to the "Y" input of a **Build Waveform** function. Set the "dt" component to the IQ rate that you obtained earlier and set an indicator to observe the waveform in the panel.
8. To observe the baseband power spectrum, connect the output of the **Quick Scale 1D** to the **Cluster Properties** "Y" component and the "dt" component to the IQ rate. The **Cluster Properties** function should already be on the template and is already connected to a **FFT Power Spectrum for 1 Chan (CDB)** node, so to view the waveform simply place the graph "Baseband Power Spectrum" on your panel.
9. Next, connect the output of the normalized signal to a **To Double Precision Complex** function, which can be found by searching the palette.

Pilots and Channel Estimation

The "Insert Pilots" function inserts a specific sequence of symbols, which is known to the receiver and is used to estimate the channel transfer function. At the receiver, you will later insert the function "Channel Estimator" which will perform this task and adjust for the phase and amplitude error introduced by the channel.

10. Connect the complex values to the **Insert Pilots** function provided, and add a **AddFrameHeader (complex)** function right after that.
11. Connect the output signal to the "data" node of the **niUSRP Write Tx Data (CDB)** function provided in the template (ensure that the input signal to the **Write Tx Data (CDB)** is a complex signal).
12. Assign the following values to the corresponding controls, and run the code.

Carrier frequency	400 MHz
IQ Rate	200 kHz (Note: This sets the value of $1/T_x$)
Gain	0 dB
Active Antenna	TX1
Symbol rate	10,000 symbols/s
Message Length	1000 bits
Pulse shaping filter	Root Raised

Tasks:

☞ Given the values set for $\frac{1}{T}$ and $\frac{1}{T_x}$, what is the corresponding value for the number of samples per symbol, L?

☞ Add the block diagram and the graphs from the front panel to your logbook (adjust the plots where necessary). Explain briefly your observations from the plots.

☞ From the spectrum plot, measure the "main lobe" bandwidth of the transmitted signal. Change the pulse shaping filter control to "none" to create rectangular pulses, and run the transmitter again. Compare the spectrum of the transmitted signal with the spectrum for root-raised-cosine pulses. Return the pulse shaping filter control setting to "Root Raised."

☞ How do the main lobe bandwidth and the spectral roll-off for root-raised cosine pulses compare with the same quantities when rectangular pulses are used?

Exercise 2: BPSK Receiver

The BPSK signal that arrives at the receiver has the form of a train of pulses, each given by

$$r(t) = \pm D g_{TX}(t) \cos(2\pi f_c t + \varphi),$$

where D is a constant (usually much smaller than the constant A in the transmitted signal), and the angle φ represents the difference in phase between the transmitter and receiver carrier oscillators. If the receiver's carrier oscillator is set to the same frequency as the transmitter's carrier oscillator, the USRP receiver will do most of the work in demodulating the BPSK signal. The receiver's **Fetch Rx Data** function will provide a train of output pulses, each given by

$$\bar{r}[n] = \pm \frac{D}{2} g_{TX}[n] e^{j\varphi}$$

The sampling rate $1/T_z$ is set by the receiver's '*IQ rate*' parameter. This rate is set to provide M samples every T seconds, where $1/T$ is the symbol rate.

The steps needed to obtain the transmitted signals are:

1. Channel Estimation: This is required to remove phase ambiguity caused by the channel and the USRP oscillators. Because the Tx and Rx channels are both using free running oscillators, there will be some ambiguous phase offset that is highly dependent on the drift and skew of the oscillators themselves. The channel estimator attempts to correct this problem by reading the received pilots, which you inserted in the Tx signal, and performing least-square channel estimation to find the channel transfer function. The inverse of the channel transfer function is then applied to the Rx signal to remove the phase offset.

2. Matched Filtering: We will use a root-raised-cosine receiver filter. This filter's impulse response $g_{RX}[n]$ is matched to the pulse shape $g_{TX}[n]$ of the transmitted pulses. The matched filter gives optimum performance in the presence of additive white Gaussian noise.

3. Pulse Synchronization: The matched filter output is an analog baseband signal that must be sampled once per symbol time, i.e., once every T seconds. Because of filtering, propagation delays, and distortion caused by the communication channel, it is necessary to determine the optimum time to take these samples. A sub-VI called **PulseAlign** has been provided to align the baseband signal.

4. Sampling: The **Decimate** function will sample the aligned baseband waveform at index 0 and every T seconds thereafter.

5. Detection: Once the baseband waveform has been sampled, each sample must be examined to determine whether it represents a symbol 1 or -1.

6. Symbol Mapping: The detected symbol values must be converted to bits.

Following the steps below, construct a BPSK receiver:

1. Calculate the '*number of samples*' for the **Fetch Rx Data**, using the message length and the symbol rate used for the transmitter. *(Hint: the 'number of samples' is equivalent to the 'number of samples' going into the **Write Tx Data (CDB)**).* Double the number of samples in a frame so that the receiver will fetch two frames of data. Since the receiver's starting point is random, this ensures that there will be one complete frame of received data in the block of samples fetched.
2. Add the **FrameSync** sub-VI after the output of the **Fetch Rx Data** function. Connect the output of **Fetch Rx Data** to the "Sampled Input" terminal of **FrameSync** and leave its remaining inputs

open. This module will find the header indicating the beginning of the Tx signal, and leave out all received symbols prior to the header, including the header.

3. Use the **Channel Estimator** function provided to eliminate phase offset. Connect the module after the **FrameSync** function.
4. To implement the receiver's matched filter, use the **MT Generate Filter Coefficients** function, just as you did for the transmitter. Set the input terminals as follows:
    a. "Modulation Type" to PSK,
    b. "pulse shaping filter" to "Root Raised Cos"
    c. "matched samples per symbol" to "M", calculated from the "actual IQ rate" $(1/T_z)$ and the symbol rate $(1/T)$ obtained from the front-panel control.
5. Extract the real part of the output of the **Channel Estimator** by using the **Complex to Real and Imaginary** function found in the palette, and convolve the real part of the signal with the "matched filter coefficients" using the **Convolution** function.
6. To observe the baseband output power spectrum, connect the output of the convolution to the "Y" component of the **Cluster Properties** node and "dt" to the IQ rate. This should already be on the template.
7. Place the **PulseAlign** sub-VI on your block diagram and wire the baseband output waveform (the output of the convolution) to the "input signal" terminal, and wire the M samples/symbol to the "receiver sampling factor" terminal.
8. Once the baseband waveform is aligned, it can be sampled with the **Decimate (single shot)** function. Note that the "decimating factor" is M.
9. Place another **FrameSync** sub-VI immediately after the **Decimate** function, using the same inputs and outputs as in step 2.
10. Next, extract the real part of the "Aligned Samples" and determine if the symbol is a 1 or a 0. You can do this by using the **Complex to Real and Imaginary** and the **Greater Than 0?** functions. You can do this in a **For Loop**.
11. Use a **Boolean to Integer** function to convert this to an integer array. Connect this array to the 'array' input terminal of an **Array Subset** function (set the 'index' input terminal to zero, and the 'length' input terminal to the length specified by the 'message length' control). Display the output of Array Subset function as 'Output bits' on the receiver front panel.
12. Measurement of the bit error rate (BER) can be automated using the **MT Calculate (After Trigger** function from the Modulation Toolkit (Analysis → Communications → Digital → Measurements subpalette). From the Configure ribbon, choose "PN Fibonacci." Set the "BER trigger threshold" to 0.4. Connect indicators to the "BER" and "trigger found?" outputs. Connect the "Output bits" to the "input bit stream" of the **MT Calculate BER After Trigger** function. When you run the program, "trigger found?" will be true whenever the measured BER is below the "BER trigger threshold".
13. Assign the following values to the corresponding controls.

Carrier frequency	400 MHz
IQ Rate	200 kHz (Note: This sets the value of $1/T_z$)
Gain	0 dB
Active Antenna	RX2
Symbol rate	10,000 symbols/s
Message Length	1000 bits
Pulse shaping filter	Root Raised

Tasks:

☞ Run the transmitter and receiver several times. What value(s) of BER do you observe?

☞ Set the Rx and Tx gain to the following settings and run each setting 5 times to obtain an average for the BER:

Tx Gain (dB)	Rx Gain (dB)
0	0
-35	-15
-37	-15
-40	-15

Exercise 3: Error Correction Coding

In this exercise, we will perform forward error correction (FEC) by adding redundancy to the transmitted information. In FEC, the transmitter sends each data bit 3 times (this makes a *triplet*). Due to the noise in the channel, the receiver might receive 8 versions of triplets, and decode the corresponding bit as shown in the table below (*majority vote decoder*).

Triplet received	Bit decoded
000	0
001	0
010	0
011	1
100	0
101	1
110	1
111	1

This implies that an error in any one of the three samples is corrected by "majority vote". To implement FEC, we will make the transmitter send 3 separate blocks of bits, i.e., 3 different arrays of same bit elements, instead of 1 block of bits. The receiver in turn will receive and demodulate these 3 blocks of bits (plus the noise introduced) separately. Afterwards, it will decode the signal and recover a single block of bits.

Following the steps below, construct a BPSK modulation system with error correction coding:

1. Duplicate the BPSK gvi you constructed in the previous exercise and rename it as **BPSK_FEC.gvi**.
2. Triple each message bit by using the **Build Array** function found in the palette. Increase the number of inputs to the function by expanding it.
3. Connect your message bits to all three inputs to the **Build Array** function and right click on the function and select "Concatenate Inputs". This will stack 3 copies of the message bits on top of each other, making a triplet.
4. At the receiver, modify the "length" node of the **Array Subset** function to 3x the message length.
5. After the **Array Subset** function, connect the "data_in" port on the **FEC Decoder** module provided to you to the output of the **Array Subset** function.
6. Connect the output of the **FEC Decoder** to the "received bits" node on the **BER function** provided to you. Connect the original message bits to the "transmitted bits" node and add an indicator to the "BER" node to observe the BER on the panel.

Tasks:

☞ Using the same system configurations as with the BPSK system without FEC, set the Rx and Tx gain to the following settings, and run each setting 5 times to obtain an average for the BER:

Tx Gain (dB)	Rx Gain (dB)
0	0
-35	-15
-37	-15
-40	-15

☞ Compare your BER results from Exercise 2 with what you obtained here. Has the FEC improved the BER of the system?

☞ What are the trade-offs involved in the error correction coding system? What are the advantages and disadvantages of the system?

Exercise 4: Differential Phase Shift Keying (DPSK)

In DPSK, the transmitter sends the difference between two adjacent bits and not the bits themselves. The table below shows how the difference is obtained for possible pairs of symbols. You can see from the table that the output of the encoded sequence is obtained by $b_n = b_{n-1} \times a_n$.

Information symbols $\{a_n\}$		1	-1	-1	1	-1	-1	1	1
$\{b_n\}$		1	1	-1	1	1	-1	1	1
Differentially encoded sequence $\{b_n\}$	1	1	-1	1	1	-1	1	1	1
Output of lowpass filter (polarity)		+	-	-	+	-	-	+	+
Decision		1	-1	-1	1	-1	-1	1	1

Note: The symbol 1 at the beginning of the encoded sequence is the reference symbol

Following the steps below, you will create a differential phase shift keying (DPSK) communication system by adding a differential encoder to your BPSK transmitter, and a differential decoder to your receiver.

1. Make new copies of your BPSK transmitter and receiver, and use the new copies to create your DPSK system.
2. Design a differential encoder and add it to the BPSK transmitter. The encoder should be added after the **AddFrameHeader** function right after the symbol mapping (the frame header also gets differentially encoded).
3. Remove the "**Insert Pilots**" and "**AddFrameHeader**" functions that were placed right before the "**niUSRP Write Tx Data**" module and simply connect the output of the "**Quick Scale 1D**" to the input data node of the "**niUSRP Write Tx Data**" module.
4. At the receiver, remove the "**Channel Estimator**" and the "**Frame Sync**" functions that were placed right after the "**niUSRP Fetch Rx Data**" module, and connect the received data directly to the **Convolution** function. Your received data is complex, therefore make sure you use the right

functions, e.g., **Convolution (CDB)** instead of Convolution **(DBL)** and **PulseAlign(Complex)** instead of **PulseAlign(real)**.

5. Design a differential decoder and add it to the BPSK receiver. Place the differential decoder immediately after the **Decimate** function, which is the receiver's sampler, and before **FrameSync**. Since the received data are complex-valued at this point, design your decoder to form the product of the current sample and the complex conjugate of the previous sample. Then take the real part of the result.

Tasks:

☞ Add the block diagram of your DPSK system to your logbook

☞ Run your code several times using the initial configuration used for the BPSK system. Explain your observation(s) on the BER performance of the system.

☞ Compare the performance of BPSK and DPSK, and explain your findings.

Appendix A: NI USRP Hardware Diagram

The NI USRP connects to a host PC creating a software defined radio. Incoming signals at the SMA connector inputs are mixed down using a direct-conversion receiver to baseband I/Q components, which are sampled by an analog-to-digital converter (ADC). The digitized I/Q data follows parallel paths through a digital down-conversion (DDC) process that mixes, filters, and decimates the input signal to a user-specified rate. The down-converted
samples are passed to the host computer.

For transmission, baseband I/Q signal samples are synthesized by the host computer and fed to the USRP at a specified sample rate over Ethernet, USB or PCI express. The USRP hardware interpolates the incoming signal to a higher sampling rate using a digital up-conversion (DUC) process and then converts the signal to analog with a digital-to-analog converter (DAC). The resulting analog signal is then mixed up to the specified carrier frequency.

The steps mentioned above can be seen in the following figure:

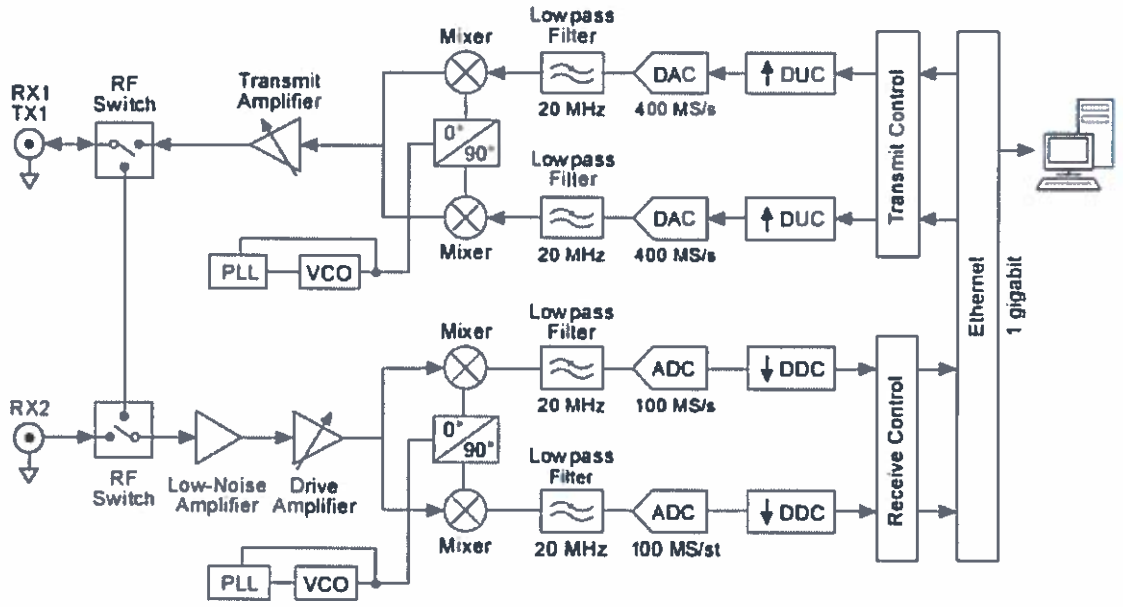


Figure A1: USRP's internal circuit.